Ansys Mechanical – PowerPoint Report Generator

IronPython 2.7 script executed directly in the **Ansys Mechanical scripting console**. It opens a WPF window in which the engineer selects the model elements (geometry, mesh, boundary conditions, contacts, results...) to include in the report, then automatically generates a PowerPoint presentation from a corporate template, archiving the extracted data as CSV along the way.

> **Language note.** This is the English translation of `README.md`. Code comments and the primary documentation (`README.md`) remain in French; this file is kept in sync with it and is the reference for English-speaking readers.

AI-assisted documentation

An AI (DeepWiki) is available to explore and understand this code:

- **Chat:** https://deepwiki.com/YannLiebherrInternship/Ansys-Report-Automation
- **Source GitHub repository:** https://github.com/YannLiebherrInternship/Ansys-Report-Automation

Table of contents

---

1. Prerequisites

Item	Detail
Ansys Mechanical	2023 or later (provides the embedded IronPython 2.7 + the `ExtAPI`/`DataModel` API)

Item	Detail
Microsoft Office	PowerPoint installed (COM Interop `Microsoft.Office.Interop.PowerPoint`)
PowerPoint template	A corporate `.pptx` file with the expected custom layouts (see §5, `00_constants.py`)
System	Windows (Windows Forms + WPF via .NET, COM Interop)

No external Python dependency is required: everything goes through the IronPython standard library (`os`, `csv`, `re`, `datetime`, `shutil`, `xml.etree.ElementTree`) and .NET assemblies loaded via `clr.AddReference()`.

2. Installation

Two methods are available. In both cases the resulting interface (see §6) is strictly identical — only the deployment and launch method changes.

	Method 1 — Manual button	Method 2 — `.wbex` extension
Installation	Manual folder copy, project by project	A single install (Extension Manager), valid for every project
File paths	Computed on first launch from a `Report Generator` folder created by hand	Handled automatically by the extension, which detects the host Workbench project itself
Access	Button "promoted" into the Automation tab, persistent across sessions of the same project	A dedicated tab, built into the extension
File cleanup	The application's "Files" tab (see §6)	The extension's "Files" tab — manual, final check required
Uninstallation	Manual deletion of the `Report Generator` folder	Automatic: the extension deletes every file associated with it

2.1 Method 1 — Manual button in the scripting console

1. Open the relevant Ansys project and save it at least once. This creates a `<ProjectName>_files` folder, which contains a `user_files` subfolder.
2. Next to `user_files` (not inside it), create a folder named exactly `Report Generator`.
3. Copy flat into this folder, with no subfolder:
   - `00_constants.py`, `01_data_export.py`, `02_image_export.py`, `03_ppt_utils.py`, `04_slides.py`, `05_interactive_slides.py`
   - `AnsysReportGenerator_WPF.py`, `AnsysReportGenerator_WPF.xaml`
   - `Master Template_def.pptx`
   - `README.md`, `README_EN.md`
   - the `logo/` folder (logo shown in the interface)

The data subfolders (`data/image_export`, `data/csv_export`, `data/reports`, `data/export_3D`) don't exist yet at this stage: they are created automatically on first launch.

4. Open Workbench, then from Workbench open Mechanical, then in Mechanical open the scripting console (**Automation** tab).
5. In the console, browse the files and open `AnsysReportGenerator_WPF.py` from the `Report Generator` folder you just created.
6. Run a first test by clicking **"Run script"**, to check that the window opens correctly.
7. If everything works, promote the script to a button: last button of the console, **"Show Button Editor"**, then **"Promote script to button"**. The button then appears in Mechanical's navigation bar, under **Automation**, and stays accessible across sessions of the same project.

> The graphical interface — and therefore this button — is currently English-only.

No path needs to be changed in the code for this to work on a new project or a new machine: all working paths are automatically recalculated from the location of `Report Generator` (`ExtAPI.DataModel.Project.ProjectDirectory`). Only the PowerPoint template must be provided manually, since it's a content file that cannot be generated automatically; if it's missing at load time, a warning is shown in the console, and generation will fail cleanly (clear message, no crash) until a valid template has been placed in the right spot. These paths can be changed without restarting the script, from the "Files" tab of the interface ("Reset" button to go back to the automatically computed values).

2.2 Method 2 — Ansys extension (`.wbex`)

- Installation via Ansys's **Extension Manager**, from the `.wbex` file.
- Once installed, the extension automatically manages its own file paths: it retrieves the destination of the Ansys Workbench project it is installed in by itself — no folder to create, no path to configure manually.
- A dedicated **"Files"** tab in the extension lets you manage these file paths and access the generated results.
- **Cleanup**: possible from that same tab, but **manual** — it requires a final check from the user before deletion; the extension never triggers this cleanup on its own.
- **Uninstallation**: on uninstall, the extension automatically deletes every file associated with it. Remember to back up any export you want to keep before uninstalling.

3. Repository structure

The folder of this repository corresponds to the contents of the `Report Generator` folder to deploy for Method 1 (§2.1).

File / folder	Role
`AnsysReportGenerator_WPF.py`	Entry point — the only file run directly in Mechanical
`AnsysReportGenerator_WPF.xaml`	Layout of the main window (see §7)
`00_constants.py` → `05_interactive_slides.py`	The six modules, loaded in order by `execfile()` (see §5)

File / folder	Role
`Master Template_def.pptx`	Corporate template — the only path not created automatically
`logo/`	Logo shown in the interface
`README.md` / `README_EN.md`	This documentation

A `data/` folder is created automatically on first launch (absent at first on a new project) and contains four subfolders:

Subfolder	Content
`image_export/`	Exported PNG images (viewport, rebuilt charts)
`csv_export/`	CSVs archived independently of PowerPoint
`reports/`	Template working copies + generated `.pptx` reports
`export_3D/`	`.avz` files (interactive 3D views), generated by the "Export to 3D" button

> The legends folder (`legend/`) is **not** inside `data/`: it lives next to it, in the Ansys project's `user_files/legend` — see §6.

`04_slides.py` and `05_interactive_slides.py` deliberately coexist: `04_slides.py` provides the original `create_..._slide` functions, which always export everything with no possible configuration, and `05_interactive_slides.py` reuses them as building blocks to construct versions filtered by the user's selection (`build_..._slides`), without duplicating the CSV/image extraction logic already written. The WPF application only calls functions from `05_interactive_slides.py`, except for `create_geometry_slide` and `create_analysis_parameters_slide` from `04_slides.py`, reused as-is.

4. Data pipeline

Generating a report, whatever the slide category involved:

1. **Extraction** — from the Mechanical tree / Tabular Data pane to CSV (`01_data_export.py`, `export_*_csv` functions, written to `CSV_EXPORT_FOLDER`), or from the 3D viewport to PNG (`02_image_export.py`, `export_current_view_image`/`export_object_image`/`export_chart_image_from_csv`, written to `IMAGE_EXPORT_FOLDER`).
2. **Building** — `PPTReportBuilder` (`03_ppt_utils.py`) first copies the template into `REPORT_OUTPUT_FOLDER` (never the original), opens this copy via COM Interop, then each call to an `add_..._slide` method adds a slide, inserting the image and/or table read in the previous steps.
3. **Saving** — once every slide has been added, the presentation is saved under its final name in `REPORT_OUTPUT_FOLDER`: this is the report delivered to the user.

The CSV is always kept on disk, regardless of whether it was successfully inserted into the PowerPoint: it remains viewable and downloadable from the "Files" tab of the interface, and constitutes an archive usable separately from the report. Its insertion as a PowerPoint table is simply skipped if the table exceeds `MAX_TABLE_ROWS`/`MAX_TABLE_COLUMNS` (`00_constants.py`), a table that large becoming unreadable once inserted into a slide.

5. Modules `00_constants.py` → `05_interactive_slides.py`

`00_constants.py`

Root paths, indexes of the template's custom layouts (`LAYOUT_IMAGE_TABLE`, `LAYOUT_TABLE_ONLY`, `LAYOUT_MESH_MULTI`), table display limits, and generic helpers independent of Ansys: `ensure_folder_exists`, `safe_file_name`, `get_unique_file_path`, `clean_cell_text`, `to_csv_cell`. Must be run first — all the constants it defines (`IMAGE_EXPORT_FOLDER`, `CSV_EXPORT_FOLDER`, etc.) are used as-is (global variables, no `import`) by every other module.

`01_data_export.py`

Everything that reads the **Tabular Data pane** or the model and writes a CSV: tabular data of an active object (`export_active_tabular_data`), contact summary, mesh report, materials used (via the Ansys `materials` module), step-parameter and solution-info tables (`export_analysis_settings_csv`/`export_solution_info_csv`, used in the "Analysis Parameters" slide).

`02_image_export.py`

Image capture of the Mechanical viewport (`export_current_view_image`, based on `ExtAPI.Graphics.ExportImage`), and "high-level" export per object type (geometry, mesh, analysis overview, any object via a `Figure` snapshot). Also contains a minimal 2D chart-drawing engine in `System.Drawing` (`export_chart_image_from_csv`): "Solution Information" trackers have no 3D representation, so their chart is redrawn from the exported CSV rather than captured from the viewport.

`03_ppt_utils.py`

`PPTReportBuilder` class: encapsulates the single COM PowerPoint session opened on the template's working copy, and exposes high-level methods to add a slide (`add_image_table_slide`, `add_table_slide`, `add_analysis_context_slide`, `add_csv_table`, `save_as`, `close`). See §10 for the details of its internal workings.

`04_slides.py`

"Legacy" `create_..._slide(report)` functions: each processes **every** object of a model category (no selection/configuration possible). Used by the UI for Geometry and Analysis Context (standalone checkboxes, no checklist).

`05_interactive_slides.py`

The largest module (~1800 lines). Provides all the support logic for the interface:

Area	Content
Cleanup	`remove_stale_figures()` — deletes leftover `Figure` objects from a previous generation
3D export (.avz)	`export_all_3d_views()` — for each analysis, exports every simple result and every child of a Contact Tool / Bolt Tool under the *Solution* branch to `EXPORT_3D_FOLDER`

Area	Content
Per-row view / section / scale / legend	`apply_view_if_exists`, `apply_section_plane`, `apply_scale_factor`, `apply_legend_if_exists` — applied right before capturing an object, then reset right after
Steps and combined slides	`evaluate_result_for_step` (step by step); `add_multi_step_image_slide` (one combined slide if a template exists for that exact step count — `MULTI_STEP_SLIDE_TEMPLATES`: 2, 3, 4, 6, or 8, otherwise automatic fallback to individual mode)
`*RowConfig` classes	`SlideRowConfig`, `GeometryPartRowConfig`, `MeshPartRowConfig`, `ContactRowConfig`, `SolutionInfoRowConfig`, `AnalysisContextRowConfig` — one instance per selection row in the UI
Collectors	`collect_views`, `collect_section_planes`, `collect_bodies`, `collect_boundary_conditions[_multi]`, `collect_bolt_pretensions[_multi]`, `collect_contact_tool_results[_multi]`, `collect_bolt_tool_results[_multi]`, `collect_all_results[_multi]`, `collect_solution_information_trackers[_multi]`, `collect_analyses`... — the `_multi` variants compile objects from **all** analyses in the project as `(object, analysis)` tuples
"Selection-aware" builders	`build_bc_slides`, `build_bp_slides`, `build_result_slides`, `build_geometry_part_slides`, `build_mesh_part_slides`, `build_contact_summary_slide`, `build_solution_info_slides`, `build_analysis_context_slides`, `build_mesh_slide` — equivalents of `04_slides.py` but limited to the checked selection
Geometry per isolated part	`isolate_body_by_transparency` — one part opaque, others semi-transparent, one slide per part
Mesh per isolated part	`show_only_body` — fully hides the other parts; up to 4 parts per slide (`LAYOUT_MESH_MULTI`), beyond that a new slide starts automatically

6. WPF interface

`AnsysReportGenerator_WPF.py` defines the **`ReportGeneratorApp`** class, which loads `AnsysReportGenerator_WPF.xaml` via `XamlReader` and drives a utility toolbar (above the tabs) and 6 vertical tabs, on the left side of the window.

Utility toolbar — 4 global actions, independent of the current selection:

Button	Action
Delete figures	Cleans up leftover `Figure` objects from a previous generation (`remove_stale_figures`)
Reset legends	Resets the viewport's legend to automatic (`reset_legend`)

Button	Action
Create basic views	Creates 7 views (X+/X-/Y+/Y-/Z+/Z-/ISO) in the View Manager, reusable in the "..." side panel (`create_basic_views`)
Export to 3D (.avz)	For each analysis, exports an interactive `.avz` 3D view of every simple result and every child of a Contact Tool / Bolt Tool under the *Solution* branch (`export_all_3d_views`, see §9), into `data/export_3D/`

Tab	Content
General slides	"Overview slides": two distinct Geometry / Mesh cards (checkbox + status + "Settings" button), "Parts to isolate (geometry)", "Mesh part to isolate", "Analysis context" (one row per project analysis, with view selection)
Conditions and contacts	Boundary Conditions, Bolt Pretension, Contacts to display, Connection: Contact Tool (*Connections* branch, no step), Solution Information
Result categories	Contact Tool Results (*Solution* branch, with steps), Results, Bolt Tool
Combined slide	Building a "different results" combined slide — see below
Report preview	One card per checked category (or added combined slide), reorderable by drag and drop — the chosen order is the report generation order
Files	Editable paths (template, images, CSV, legends, reports), data folder cleanup (see below), list of already-generated CSVs (Open/Show in folder), progress + access to the last generated report

"Combined slide (different results)" tab. This flow used to live in 3 successive modal dialog boxes (template choice, then grid, then result choice); it is now fully integrated into this tab, with no separate window at all.

- At the top: choosing a multi-image template (2/3/4/6/8 results, same `MULTI_STEP_SLIDE_TEMPLATES` as the multi-step combined slides) and an "Add to report" button.
- On the left: a 2×4 grid where only the first N cells of the chosen template are active; clicking an empty cell shows, on the right, the (filterable) list of available results.
- On the right: clicking a result switches the panel to its full graphic configuration (same fields as a normal row — view/section/legend/appearance/scoping/scale factor — but with no notion of step, a different, fixed result per cell); the "Apply" button confirms the cell.

"Add to report" requires all active cells to be configured, then adds the configuration (`MultiResultSlideConfig`) to `self._multi_result_slides` and resets the grid to build another one — nothing is generated immediately: like the other categories, the slide appears as a card in the "Preview" tab (dedicated "Delete" button, no checkbox) and is only built when clicking "Generate report" (`ReportGeneratorApp._build_multi_result_tab`, the `_on_multi_result_*`/`_show_multi_result_*` methods, `build_multi_result_slide`/`capture_multi_result_cell_image` in `05_interactive_slides.py`).

Global side configuration panel ("..."). Every selection row has a **"..."** button that no longer opens a separate window: it displays, to the right of the main window, a "SETTINGS" panel shared by all tabs. Its content depends on the "kind" of the clicked row (`ReportGeneratorApp._open_config_panel`):

Kind	Fields
`"result"`	View / section / legend (file + orientation) / color display mode (Contour View) / scoping display / deformation scale (manual, or Auto Scale x1/x2) / step selection
`"geometry_part"`	View / section / context opacity (isolated part in geometry)
`"mesh_part"`	View only (isolated part in mesh, but also Geometry/Mesh/Analysis Context)
`"solution_info"`	Title / axes / color of the rebuilt chart

Each field category is a pair of shared functions `_build_*_fields`/`_apply_*_fields`, which set/read their controls on a generic `target` (`_ConfigFieldsHolder`, a simple attribute bag) rather than on `self` of a dedicated window class — this decoupling lets the same code serve both the global side panel and the cell panel of the "Combined slide" tab. "Apply" confirms (`row_config.configured = True`) and closes the panel; "Cancel"/the "x" button close without confirming.

This same panel can also open in **bulk mode**: every section header offers a "Configure selection..." button that applies the chosen settings to every checked row of the section at once (`ReportGeneratorApp._on_bulk_config_click`/`_open_config_panel(..., bulk_rows=...)`) — a plain Python loop over the `row_config` objects, with no panel rebuild or API call per row.

View selection for Geometry / Mesh / Analysis Context. These three checkboxes have a selectable (View Manager) view: for Geometry and Mesh, a "..." button opens the global side panel in `"mesh_part"` mode on a dedicated `MeshPartRowConfig` (`self._geometry_view_config`/`self._mesh_view_config`); for Analysis Context, `AnalysisContextRowConfig` carries a `view_name` with its own "..." button. In all three cases, the chosen view is applied right before capture (`apply_view_if_exists`), with no reset afterward.

Result appearance (Contour View / legend / scoping / deformation scale). The panel in `"result"` mode exposes four settings per row:

- **Contour View** (`ResultPreference.ContourView`): `ContourBands`, `Isolines`, `SmoothContours`, `SolidFill` — .NET names kept as-is in the UI.
- **Legend orientation** (`GlobalLegendSettings.LegendOrientation`): Vertical or Horizontal.
- **Scoping display** (`ResultPreference.ScopingDisplay`): `ScopedBodies` (default), `ResultOnly`, `AllBodies`.
- **Deformation scale**: manual (numeric factor) or one of the two native "Auto Scale x1"/"Auto Scale x2" presets (`ResultPreference.DeformationScaling`/`DeformationScaleMultiplier`).

By default, an unconfigured result is captured with `ContourBands`, vertical legend, `ScopedBodies` scoping, manual x1 scale. These settings are carried by `SlideRowConfig` and applied only while capturing the relevant row, then systematically reset right after (`reset_contour_view`/`reset_legend_orientation`/`reset_scoping_display`/`reset_scale_factor`), so that a setting chosen for one row never "leaks" onto the next one. The `apply_*` functions call `ExtAPI.Graphics.Redraw()` right after changing their property: without this explicit call, the image exported right after would keep reflecting the old state.

Two settings, on the other hand, apply globally, to every image export without exception: `ModelColoring = ModelColoring.ByMaterial` (`set_material_display`, before every geometry/mesh export), and `ShowLogo = False` (forced in `export_current_view_image`, so that no report image shows the Ansys logo).

Camera framing: the user's responsibility. The image-export functions no longer call `ExtAPI.Graphics.Camera.SetFit()` before capture: this would silently overwrite any view chosen by the user. It is therefore up to the user to frame the view (manually or via a named view) before generating the report. Only `create_basic_views()` (the "Create basic views" button) still uses `SetFit()`, since its purpose is to define the framing of the 7 standard views.

Generation (`ReportGeneratorApp._on_generate`) walks through the order of the "Report preview" tab, opens a single `PPTReportBuilder` session, calls the `build_..._slides` function matching each category, updates the progress bar (`SWF.Application.DoEvents()` to keep the window responsive), then cleanly closes the PowerPoint session and enables the "Open"/"Show in folder" buttons.

Files tab: 4-quadrant layout. Top-left: file paths (the Template highlighted, a "sensitive" path the whole generation depends on; the Legends line with a "to check" note, see below). Top-right: data folder cleanup (see below). Bottom-left: CSV list. Bottom-right: progress + access to the last report. The "Generate report"/"Close" buttons stay at the window level, visible regardless of the active tab.

Legends folder: moved out of `DATA_ROOT`. `LEGEND_FOLDER` (`00_constants.py`) points to `<project>/user_files/legend` rather than `data/legend`. This folder is no longer created automatically by the script — it is maintained manually by the engineer in the Ansys project's files, this script only *reads* it. A console warning flags its absence at load time, just like for the template. No longer being inside `DATA_ROOT`, it no longer appears in the cleanup tiles.

Data folder cleanup. Tiles are generated dynamically from `list_data_cleanup_folders()` (`00_constants.py`), which lists every subfolder of `DATA_ROOT` except `LEGEND_FOLDER` — a new data subfolder added to the code automatically gets its own cleanup tile, with no UI change needed. Each tile's size and file count come from `get_folder_stats()`, a recursive walk (`os.walk`) recomputed when the app opens and after every generation step (`_update_generation_progress`) — its cost grows with the number of files present. `clear_folder_contents()` deletes a folder's contents without deleting the folder itself, which avoids having to recreate it (`ensure_folder_exists`) on the next generation. The per-tile "Clear" button and the global "Delete all" button both call this same function — a single deletion implementation. Both actions are gated behind a confirmation dialog (`MessageBoxButton.YesNo`), the deletion being irreversible (no trash/undo). If the reports folder is cleared, the last-report status tile goes back to its neutral state (`_reset_report_status_tile`), since the file it referenced no longer exists.

CSV and report lists: "Show in folder" rather than a download. The CSV tile grid has become a tabular list (one row per file, name on the left + buttons on the right). The download buttons (`SaveFileDialog`) have been removed everywhere, in favor of a **"Show in folder"** button (`_on_show_in_folder`, `Process.Start("explorer.exe", "/select,\"<path>\"")`), which opens Windows Explorer with the file already selected. The viewing button is called "Open" (`Process.Start(path)`).

> `AnsysReportGenerator_WPF.xaml` only contains declarative layout: check that file directly for the exact appearance, or §7 below for how it works.

7. XAML: declarative layout and its link to Python

What XAML is here. `AnsysReportGenerator_WPF.xaml` only contains declarative layout: styles, colors (`Brush`), and the fixed controls of the window (tabs, cards, toolbar buttons...), each identified by an `x:Name`. Unlike a "classic" WPF project (compiled XAML tied to an `x:Class` and an auto-generated code-behind file), this project loads the XAML **at runtime** via `XamlReader.Load`: no compilation, no partial class, no

`Click="..."` attribute anywhere in the XAML. All the logic — dynamic list building, event wiring — is written in Python.

The XAML ↔ Python link. Every control named in the XAML is looked up on the Python side via `self.window.FindName("ExactName")`, centralized in `ReportGeneratorApp._find_controls` (e.g. `btnGenerate` → `self.btn_generate`). Event handlers are then wired explicitly in Python (`ReportGeneratorApp._wire_events`), e.g. `self.btn_generate.Click += self._on_generate` — none of this appears in the XAML.

> **Pitfall: `ControlTemplate` `NameScope`.** An element defined inside a `ControlTemplate` (a `Style`) lives in a `NameScope` separate from the window's: `FindName()` cannot see it, even with a correctly set `x:Name`. This is the case for the credit-card logo (bottom of the tab column, defined inside the `TabControl` style's `ControlTemplate`): it is resolved via a `DynamicResource` (`SidebarLogoBitmap`) instead of an `x:Name`, the resource being populated from Python (`self.window.Resources["SidebarLogoBitmap"] = bitmap`, see `_load_logo`) — a `ResourceDictionary` stays accessible like a plain dictionary regardless of where in the visual tree it's defined.

Adding a new button, step by step:

1. **Find a spot** in the XAML based on a similar existing button: a toolbar button (`btnDeleteFigures`, `btnResetLegends`...), a section-header button (`btnZoneCheckXxx`/`btnZoneConfigXxx`), or a bottom-of-window button (`btnGenerate`/`btnClose`).
2. **Add the element** inside the relevant `StackPanel`/`DockPanel`, reusing an existing style rather than defining a new one:

   ```
   <Button x:Name="btnMyButton" Content="My action" Style="{StaticResource SecondaryButton}"/>
   ```

   Available styles: `PrimaryButton`, `SecondaryButton`, `MiniButton`, `DangerButtonLight`, `DangerButtonStrong`.
3. **Validate the XAML** before testing in Mechanical — a markup error fails in a not-very-explicit way at load time:

   ```
   [xml]$doc = Get-Content -Raw "AnsysReportGenerator_WPF.xaml"
   ```

4. **Retrieve the control on the Python side**, in `_find_controls`:

   ```
   self.btn_my_button = self.window.FindName("btnMyButton")
   ```

5. **Write the handler**, then **wire it** in `_wire_events`:

   ```
   def _on_my_button_click(self, sender, e):
       ...
   ```

```
self.btn_my_button.Click += self._on_my_button_click
```

8. Ansys domain concepts used in the code

Term	Meaning	Where in the code
Step / Load Case	A loading step in an analysis (e.g. Step 1 = preload, Step 2 = service load)	`get_step_count`, `selected_steps`, `evaluate_result_for_step`
Boundary Condition (BC)	An imposed constraint/load (fixed support, pressure, force...)	`collect_boundary_conditions[_multi]`, `build_bc_slides`
Bolt Pretension	Bolt preload	`collect_bolt_pretensions[_multi]`, `build_bp_slides`
Contact Tool	Analysis of a contact's quality (gap, pressure, slip...) — exists twice: one under *Connections* (definition, no step) and one under *Solution* (results, with steps), distinguished by their position in the tree (`_is_descendant_of`)	`collect_contact_tool_results` vs `collect_connection_contact_tool_results`
Bolt Tool	Forces in bolted connections (axial, shear...)	`collect_bolt_tool_results[_multi]`
Solution Information	Solver convergence data; its children ("trackers") only have a 2D chart, no 3D view	`collect_solution_information_trackers`, `export_chart_image_from_csv`
Named View	A camera view saved in the View Manager	`collect_views`, `apply_view_if_exists`
Section Plane	A cutting plane to reveal the model's interior	`collect_section_planes`, `apply_section_plane`
Focus	An aggregated result filtered by a selection (not yet integrated into the active UI)	—

9. Ansys Mechanical APIs used

Access to the model

```
ExtAPI.DataModel.Project.Model                  # model root
ExtAPI.DataModel.Project.Model.Analyses         # list of the project's analyses
ExtAPI.DataModel.AnalysisList                    # same, equivalent shortcut
```

```
ExtAPI.DataModel.GetObjectsByType(DataModelObjectCategory.XXX)  # search by
category across the whole tree (BC, Bolt Pretension, Contact Tool, Bolt Tool,
Contact Region, Figure...)
ExtAPI.DataModel.Project.Model.GetChildren(DataModelObjectCategory.Body, True)  #
every body (True = recursive)
ExtAPI.DataModel.Project.Model.Geometry        # Geometry root, carried by
MeshPartRowConfig for the Geometry checkbox (see §6)
ExtAPI.DataModel.Project.Model.Mesh            # Mesh root, carried by
MeshPartRowConfig for the Mesh checkbox (see §6)
ExtAPI.DataModel.Project.ProjectDirectory      # "<ProjectName>_files" folder of
the current project - used to locate PROJECT_DIR (§2)
```

Display and image capture

```
ExtAPI.Graphics.Camera.SetFit()                # frames the camera on the active
object - used only by create_basic_views() (see §6)
ExtAPI.Graphics.ExportImage(path, GraphicsImageExportFormat.PNG, settings)  #
settings = Ansys.Mechanical.Graphics.GraphicsImageExportSettings()
ExtAPI.Graphics.ViewOptions.ModelColoring      # material-based coloring
ExtAPI.Graphics.ViewOptions.ShowMesh           # mesh display
ExtAPI.Graphics.ViewOptions.ShowLogo           # Ansys logo - always disabled (see
§6)
ExtAPI.Graphics.ViewOptions.ResultPreference.ContourView              # result
color display mode (§6) - read/written via getattr() (§11)
ExtAPI.Graphics.ViewOptions.ResultPreference.ScopingDisplay           # scoping
display mode (§6)
ExtAPI.Graphics.ViewOptions.ResultPreference.DeformationScaling       #
deformation scale mode (Auto/UserDefined)
ExtAPI.Graphics.ViewOptions.ResultPreference.DeformationScaleMultiplier  # scale
factor (manual, or x1/x2)
MechanicalEnums.Graphics.ScopingDisplay        # matching enum: ScopedBodies /
ResultOnly / AllBodies
MechanicalEnums.Graphics.DeformationScaling    # matching enum: Auto / UserDefined
ExtAPI.Graphics.GlobalLegendSettings.LegendOrientation  #
LegendOrientationType.Vertical / .Horizontal
ExtAPI.Graphics.ImportLegend(path, unit)       # applies an .xml legend file - the
unit must match that of the object CURRENTLY ACTIVE (systematic Activate() right
before)
ExtAPI.Graphics.ModelViewManager.ExportModelViews(path)  # lists named views, to
XML
ExtAPI.Graphics.ModelViewManager.ApplyModelView(view)    # activates a named view
ExtAPI.Graphics.ModelViewManager.Capture3DImage(path)    # exports a .avz
(interactive 3D view) of the active object - "Export to 3D" button
ExtAPI.Graphics.SectionPlanes                  # available cutting planes
(apply_section_plane)
ExtAPI.Graphics.Redraw()                       # forces the viewport to refresh -
required after any scripted display-property change
```

Results and steps

```
SetDriverStyle.ResultSet                        # + .SetNumber: repositions a
result on a given step before re-evaluation (evaluate_result_for_step)
obj.Activate()                                   # activates the object in the
viewport - a prerequisite for most captures/exports
obj.Name / obj.Children / obj.Parent / obj.DataModelObjectCategory  # available on
most individual objects
obj.AddFigure()                                  # then figure.Activate(): Figure
snapshot, a reliable capture preferred over a direct "live" one
```

Other notable APIs

```
Ansys.ACT.Mechanical.Transaction   # "with Transaction(True): ..." - defers UI
refresh during bulk operations (deleting figures, looping over every body...)
materials.GetMaterialPropertyByName(material, group)   # Ansys module to read
material properties
```

.NET / COM side (outside the Ansys API)

```
clr.AddReference("Microsoft.Office.Interop.PowerPoint")  # + "Office"
clr.AddReference("System.Windows.Forms") / "System.Drawing"
clr.AddReference("PresentationFramework") / "PresentationCore" / "WindowsBase"  #
WPF
```

10. How the code drives PowerPoint (COM Interop)

The project doesn't depend on any Python library to manipulate PowerPoint: `python-pptx` (like `pandas` or `openpyxl`) is incompatible with IronPython 2.7, the Python engine embedded in Ansys Mechanical, and is therefore never used here. It drives the PowerPoint application installed on the machine directly via **COM Interop**: Microsoft Office exposes a COM API, and .NET provides "Interop" assemblies (`Microsoft.Office.Interop.PowerPoint`, `Office`) that translate this COM API into .NET classes usable from any .NET language — hence from IronPython, which itself runs on the .NET CLR. This is what `03_ppt_utils.py` does right at the top of the file:

```
clr.AddReference("Microsoft.Office.Interop.PowerPoint")
clr.AddReference("Office")
import Microsoft.Office.Interop.PowerPoint as PPT
import Microsoft.Office.Core as Office
```

`clr.AddReference` loads the matching .NET assembly (installed with Office, independently of the project), after which `PPT` and `Office` are used like ordinary Python modules — except that every object handled (`Presentation`, `Slide`, `Shape`...) is actually a remote COM object: every property access or method call actually goes and queries the running PowerPoint process, which has a cost (hence several optimizations described further below).

All the logic is concentrated in the `PPTReportBuilder` class, which owns a single PowerPoint session for the entire report generation (one open/close, not one per slide). Its constructor illustrates the module's central principle:

```
def __init__(self, template_path):
    self.working_copy_path = get_unique_file_path(
        REPORT_OUTPUT_FOLDER, _build_working_copy_base_name(), ".pptx")
    shutil.copyfile(template_path, self.working_copy_path)

    self.app = PPT.ApplicationClass()
    self.app.Visible = True
    self.presentation = self.app.Presentations.Open(self.working_copy_path,
WithWindow=True)
```

`PPT.ApplicationClass()` starts (or retrieves) an instance of the PowerPoint application itself, exactly as if the user had double-clicked its icon; `self.app.Presentations.Open(...)` then opens a file in it, returning a `Presentation` object which every subsequent operation acts on. The original template is never opened directly: a copy (`working_copy_path`) is created right before via `shutil.copyfile`, and it's this copy that gets opened — an accidental `Ctrl+S` in the PowerPoint window during generation therefore overwrites the copy, never the corporate template. `self.app.Visible = True` is not cosmetic: a session left invisible turned out to be unstable on a report with many slides (the `SlideMaster` object would eventually become inaccessible mid-generation), so the PowerPoint window stays visible throughout generation and closes normally at the end, in `close()`.

Adding a slide always consists of requesting a custom layout from the template by its index, then appending this slide at the end of the presentation:

```
def _add_slide(self, layout_index):
    layout = self.presentation.SlideMaster.CustomLayouts[layout_index]
    return self.presentation.Slides.AddSlide(self.presentation.Slides.Count + 1,
layout)
```

`SlideMaster.CustomLayouts` is the list of custom layouts defined in the template (visible in PowerPoint via View > Slide Master); their index (`LAYOUT_IMAGE_TABLE = 10`, etc., in `00_constants.py`) is determined once and for all by listing the template's layouts (see §13) and doesn't change again as long as the template isn't modified. `add_image_table_slide` then illustrates how a slide's zone is filled once it's been created:

```
slide.Shapes[8].TextFrame.TextRange.Text = comment
slide.Shapes[2].TextFrame.TextRange.Text = title
...
coord = self.presentation.SlideMaster.CustomLayouts[LAYOUT_IMAGE_TABLE].Shapes[3]
slide.Shapes.AddPicture(img_path, Office.MsoTriState.msoFalse,
Office.MsoTriState.msoTrue,
                        coord.Left, coord.Top, coord.Width, coord.Height)
```

Each `Shapes[n]` corresponds to a precise zone defined in the layout at the time it was created in PowerPoint (a title zone, an image zone, a table zone...); the order and index of these zones are fixed by the template, not by the code, hence the importance of never rearranging the zones of an existing layout without updating the indexes used in `03_ppt_utils.py` (see §13). Text is always assigned on the newly created slide, never on the layout itself: modifying the layout would modify the master template for every future slide. To position the image, the code looks up the coordinates (`Left`, `Top`, `Width`, `Height`) of the image zone as defined *in the layout*, rather than hard-coding these coordinates: the image's position and size therefore stay consistent with what was drawn in the template, even if it evolves.

`add_csv_table` is the most performance-sensitive part, since every statement in the following block is a COM round-trip:

```
for r in range(1, rows + 1):
    row_cells = table.Rows(r).Cells
    for border_index in range(1, 5):
        row_cells.Borders(border_index).ForeColor.RGB = 0x000000
        row_cells.Borders(border_index).Weight = 1
```

Borders are applied once per whole row (`table.Rows(r).Cells` accepts a range of cells) rather than cell by cell × side by side, which divided a table's formatting time by roughly the number of columns (up to 45 seconds for 8 rows before this optimization, versus a fraction of a second after). Text and font, on the other hand, have no "per-range" equivalent in PowerPoint's COM API and therefore necessarily remain applied cell by cell in the following loop. One last quirk: after filling the table, the code forces `table.Rows(r).Height = 1` on every row — a deliberately absurd value, but PowerPoint automatically brings it back to the minimum height actually needed to fit the text, which is the only way to tighten a table already created (`AddTable` allocates a much larger height than needed for size-7 text by default, which would overflow the slide without this fix).

Finally, `close()` illustrates the rule to systematically follow with COM objects: release them explicitly rather than counting on Python's garbage collector, so as to never leave an invisible PowerPoint process running in the background after an error:

```
def close(self):
    self.presentation.Save()
    self.presentation.Close()
    self.app.Quit()
```

11. Python design choices used in the project

Several recurring choices in the code answer constraints specific to IronPython 2.7 and to execution in the Mechanical scripting console; understanding them helps in reading (and extending) any module of the project.

Loading via `execfile()` rather than `import`. `AnsysReportGenerator_WPF.py` does not do `import constants` or `from data_export import ...`: it calls `execfile(module_path)` for each of the six modules, in order. `execfile()` executes a file's contents as if it had been typed directly next in the same

console, in the same global namespace — unlike `import`, which would create a separate namespace (`data_export.export_active_tabular_data` instead of `export_active_tabular_data`). It's this deliberate sharing of a single global namespace that lets `05_interactive_slides.py` call `export_active_tabular_data` (defined in `01_data_export.py`) directly with no prefix, exactly as the Mechanical scripting console itself does with `ExtAPI`/`DataModel`. It's also what lets a function defined earlier reference a name defined later in another module: `00_constants.py` uses `PROJECT_DIR`, which is actually only defined in `AnsysReportGenerator_WPF.py`, *before* the `execfile()` of `00_constants.py` — the loading order (`00` → `05`, then the main script last in the console) is therefore significant and must never be changed.

Accessing .NET enums via `getattr()` rather than an explicit import. Several places in the code, for instance `apply_contour_view` in `05_interactive_slides.py`, write:

```
vo.ResultPreference.ContourView = getattr(vo.ResultPreference.ContourView,
contour_view)
```

instead of importing the matching `.NET` enumeration and writing a long `if/elif` chain to convert the string chosen in the UI (`"ContourBands"`, `"Isolines"`...) into an enum value. `getattr(obj, "MemberName")` looks up the attribute named `"MemberName"` on the *type* of `obj` (here the type of the `ContourView` enum already present on the current instance): since the strings used in the UI's dropdowns (`CONTOUR_VIEW_OPTIONS`) carry exactly the same names as the .NET enum's members, `getattr` performs the string → enum-value conversion directly in one line, without having to explicitly import each enum type or keep it up to date if Ansys adds a member in a future version.

Local failure, never an exception bubbling up to the UI. Almost every export or setting-application function follows the same pattern:

```
try:
    ...
except Exception as e:
    print "Error: " + str(e)
    return False  # or None
```

This choice is deliberate: a report generation can span dozens of slides, and a single misconfigured Boundary Condition row (or an image that fails to export) must not interrupt the entire generation and lose the work already done on previous slides. The error is therefore absorbed locally, logged to the scripting console (visible to the engineer), and the function returns a "neutral" value (`False`, `None`, or simply does nothing) that the caller can test to decide whether to continue.

"collect_" functions always return a plain Python list. Whether the source is `ExtAPI.DataModel.GetObjectsByType(...)`, a recursive tree walk, or the compilation of several analyses (`_multi` variants), every collector returns an ordinary Python `list`, never the original .NET/COM object. This fully decouples the WPF interface (which builds its dropdowns and checklists from these lists) from the details of how each object category is found in the Mechanical tree — a new collector can entirely change its internal logic without the UI code consuming it having to change.

Constants and `(label, value)` options. The options meant to appear in the UI (`CONTOUR_VIEW_OPTIONS`, `LEGEND_ORIENTATION_OPTIONS`, `DEFORMATION_SCALE_MODE_OPTIONS`, `BASIC_VIEW_ORIENTATIONS`...) are systematically lists of tuples `(label shown in the UI, technical value used in the code/API)`, with symmetrical `xxx_label`/`xxx_from_label` functions to convert one way or the other. This cleanly separates what is shown to the engineer (safely editable) from what must stay identical to the exact name expected by the .NET API.

12. Python fundamentals illustrated by the project's code

This section revisits the basic building blocks of the Python language (IronPython 2.7-compatible) using real examples from the project, for a reader discovering Python through this code.

Variables and types. A variable has no declared type, it takes the type of whatever is assigned to it: `DATA_ROOT = os.path.join(PROJECT_DIR, "data")` creates a `DATA_ROOT` variable of type `str`. `MAX_TABLE_ROWS = 50` creates an integer. A list is written between square brackets and can grow dynamically: `_MODULE_FILES = ["00_constants.py", "01_data_export.py", ...]`. A dictionary maps keys to values between curly braces; `_DEFAULT_FILE_PATHS = dict((name, globals()[name]) for name, _, _, _ in FILE_PATH_SETTINGS)` builds one on the fly from a list of tuples.

Functions. `def` defines a function, its parameters go between parentheses, and `return` gives back its output value (a function with no `return` implicitly returns `None`):

```
def safe_file_name(name):
    return re.sub(r'[\\/:*?"<>|]', "_", name).strip() or "object"
```

A parameter can have a default value, used if the caller doesn't supply one: `def add_image_table_slide(self, title, subtitle, img_path=None, csv_path=None, comment=" ")` can therefore be called with only `title` and `subtitle`, `img_path` then being `None`. The project systematically uses strings formatted with `.format()`, never Python 3.6+ f-strings (unavailable in IronPython 2.7): `"result_{}.csv".format(step_id)` rather than `f"result_{step_id}.csv"`.

Classes. `class ClassName(object):` defines a class (the explicit `(object)` is required in Python 2 to get a "new-style" class). `__init__` is the constructor, called automatically when an instance is created; `self` (the first parameter of every method) refers to the instance itself and must be used explicitly to read or write an attribute:

```
class PPTReportBuilder(object):
    def __init__(self, template_path):
        self.working_copy_path = get_unique_file_path(...)
        self.app = PPT.ApplicationClass()

    def save_as(self, output_path):
        self.presentation.SaveAs(output_path)
```

`self.app` and `self.working_copy_path` are attributes belonging to each `PPTReportBuilder` instance: two instances created separately would each have their own PowerPoint session, without interference. A method is

then called on an instance: `builder = PPTReportBuilder(TEMPLATE_PATH)` then `builder.save_as(output_path)`.

Loops and conditions. `for` iterates over any sequence (list, range of numbers, result of an API query); `range(1, rows + 1)` produces the integers from 1 to `rows` inclusive (Python always excludes `range`'s upper bound). `if`/`elif`/`else` tests conditions; indentation (always 4 spaces in this project, never mixed tabs) delimits blocks, there are no curly braces in Python:

```
for step_id in steps:
    if step_id in selected_steps:
        rows.append(evaluate_result_for_step(obj, step_id))
    else:
        print "Step skipped: {}".format(step_id)
```

Error handling (`try`/`except`). A `try` block runs potentially risky code (COM access, disk access, Ansys API call); if an exception is raised, execution jumps directly to the matching `except` block rather than crashing the whole script:

```
try:
    graphics.ExportImage(output_path, export_settings)
    return True
except Exception as e:
    print "Error exporting view: {}".format(e)
    return False
```

`except Exception as e` catches any standard error and makes it available in the `e` variable (usually converted to text via `str(e)` to display it). This pattern is everywhere in the project (see §11).

Context managers (`with`). `with open(path, "rb") as f:` opens a file and guarantees it's closed automatically on exiting the block, even if an error occurs inside — a safer, shorter equivalent of a manual `try`/`finally` with `f.close()`. Used for every CSV file access in the project, and repurposed for a different use with `with Transaction(True): ...` (`Ansys.ACT.Mechanical.Transaction`), which doesn't manage a file but defers Mechanical's UI refresh until the block exits, to speed up bulk operations.

List comprehensions. A comprehension builds a new list in a single expression, more concise than a classic `for` loop with `append`: `[cell.decode("utf-8") for cell in row]` (in `add_csv_table`) reads back each cell of a CSV row and decodes it from UTF-8, directly producing the decoded list.

`import` vs `execfile`. The project uses `import` for standard modules (`import csv`, `import os`) and .NET assemblies (`import Microsoft.Office.Interop.PowerPoint as PPT`, after `clr.AddReference`), but `execfile()` to load its own `00` through `05` modules — see §11 for the explanation of this unusual choice, specific to the execution context in the Mechanical console.

13. Creating a new custom slide in the Master Template

Adding a new slide type to the report first requires creating the matching layout in the PowerPoint template itself, and only then writing the Python code that fills it in. The PowerPoint-side procedure is strict on one

point: **the new layout must always be inserted at the end of the slide master, never in the middle**. Every index used in the code (`LAYOUT_IMAGE_TABLE = 10`, `LAYOUT_TABLE_ONLY = 8`, `LAYOUT_MESH_MULTI = 11`, in `00_constants.py`) corresponds to the layout's position in the template's `CustomLayouts` list; inserting a new layout in the middle of that list shifts the index of every existing layout after it, and silently breaks every slide already generated by the current code.

To create the layout, open the Master Template in PowerPoint (View > Slide Master), insert a new layout after the existing ones, and build its content either by drawing new zones (text box, image zone, table), or by copying elements from an existing layout close to what's needed. Once the layout is finished, save this new version of the template under a **different name** from the original (for instance by adding a suffix), to keep a backup copy of the template currently used in production in case the change turns out to be incompatible with the existing code.

The index of the new layout must then be identified, along with the index of each of its zones (`Shapes`), since it's by these indexes that the Python code refers to them (see `Shapes[n]` in §10). This is done by running, in the Mechanical scripting console, a small script that opens the template via COM Interop exactly like `PPTReportBuilder` does, and lists the available layouts:

```
import clr
import os
import System

clr.AddReference("Microsoft.Office.Interop.PowerPoint")
clr.AddReference("Office")
import Microsoft.Office.Interop.PowerPoint as PPT
import Microsoft.Office.Core as Office
from Microsoft.Office.Core import MsoTriState

app = PPT.ApplicationClass()
app.Visible = True
template_path = r"PATH_TO_THE_TEMPLATE.pptx"  # adjust as needed
presentation = app.Presentations.Open(template_path, WithWindow=True)
custom_layouts = presentation.SlideMaster.CustomLayouts

for design in presentation.Designs:
    for i in range(1, design.SlideMaster.CustomLayouts.Count + 1):
        layout = design.SlideMaster.CustomLayouts[i]
        print(i, layout.Name)
```

This first block prints the full list of existing layouts with their index and name (for example `(1, "Title Page")`, `(10, "Image + Table")`...): that's where the index assigned to the newly added layout can be spotted. Once that index is identified, select that layout then list its zones in the order PowerPoint knows them:

```
slide = custom_layouts[10]  # replace with the new layout's index

index = 0
for shape in slide.Shapes:
```

```
    index += 1
    print shape.Name
```

This second block gives, for each zone of the layout, its name and its position in the `Shapes` collection (the first item listed corresponds to `Shapes[1]`): it's this mapping between position and the zone's visual role (title, image, table, comment...) that must then be carried over into the Python code, exactly as `LAYOUT_IMAGE_TABLE` is documented today as a comment in `00_constants.py` (`# title[2] / subtitle[4] / image[3] / table[1] / comment[8]`). A new `add_..._slide` function can then be added to `03_ppt_utils.py` following the model of `add_image_table_slide`, using the new layout's index and the zone indexes identified this way.

14. Known pitfalls / technical choices

- **IronPython 2.7 constraints**: the scripting engine embedded in Ansys Mechanical runs Python 2.7 via .NET, not Python 3. Any code change must therefore stay compatible with these restrictions:
  - `.format()` instead of f-strings: `"result_{}.csv".format(step_id)`, never `f"result_{step_id}.csv"` (syntax error in IronPython 2.7).
  - `print "text"` as a statement, never `print("text")` as a function.
  - `os.path.join(...)` instead of the `pathlib` module, absent from IronPython 2.7.
  - No type annotations (`variable: str = ""`), no `async`/`await`.
  - The `pandas`, `openpyxl`, and `python-pptx` libraries are incompatible and must never be imported — this is why every piece of tabular data in the project goes through plain CSV files (standard `csv` module) rather than these libraries.
- **PowerPoint session always visible** (`self.app.Visible = True` in `PPTReportBuilder.__init__`): a session left invisible turned out to be unstable on a report with many slides (`COMException` on `SlideMaster` mid-generation). The PowerPoint window closes normally at the end (`close()`).
- **The original template is never opened directly** — always a working copy (see §4), to never risk overwriting it via an accidental `Ctrl+S` during generation.
- **Table borders applied per whole row**, not cell by cell × side by side: every COM round-trip is expensive, this optimization divided a table's formatting time by roughly N (N = number of columns).
- **Legend unit always deduced dynamically** (`get_result_display_unit`, reads the text shown in `VisibleProperties`, not `result_obj.Maximum.Unit` which was found unreliable): `ImportLegend()` compares the requested unit to that of the object **currently active** in the viewport, hence a systematic explicit `Activate()` right before, to avoid a one-row offset with the object actually being processed.
- **CSV always read/written in explicit UTF-8** (`open(path, "rb")` + manual decoding): units returned by Mechanical sometimes contain special characters (degree, micro...) that crash a read/write without explicit encoding.
- **Table display limit** (`MAX_TABLE_ROWS` / `MAX_TABLE_COLUMNS`, 50×50 by default): beyond that, the CSV is still generated but not inserted as a PowerPoint table (unreadable once inserted).
- **Inserting a layout into the template**: always at the end of the slide master, never in the middle — see §13 for the full procedure and the reason (index shift of the `LAYOUT_*` constants used throughout the code).